Methods

1. Empirical Mode Decomposition

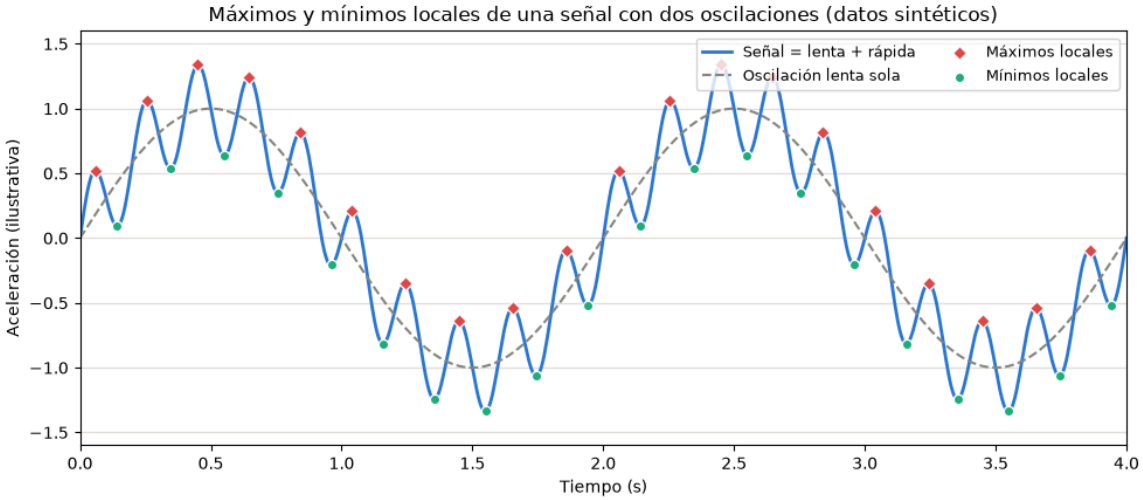
- Proposed by Huang et al. (1998) in "The Empirical Mode Decomposition and the Hilbert Spectrum for Nonlinear and Non-Stationary Time Series Analysis."
- Descompone la señal en Intrinsic Mode Functions (IMFs) por sifting de la señal más rápida a las mas lentas.

Process:

1. Identify all local maxima and minima of the signal.
2. Connect the maxima with a cubic spline to form the upper envelope, and the minima to form the lower envelope.
3. Compute the mean of the two envelopes.
4. Subtract this mean from the signal.
5. Repeat steps 1 to 4 on the result until it satisfies the two IMF conditions. This is the first IMF.
6. Subtract the IMF from the original signal and repeat the whole process on the remainder to extract the next IMF.

IMFs' conditions:

1. La suma de los maximos y minimos menos los cruces por cero es igual o menor que uno.
2. Media local igual a cero en cada instante.



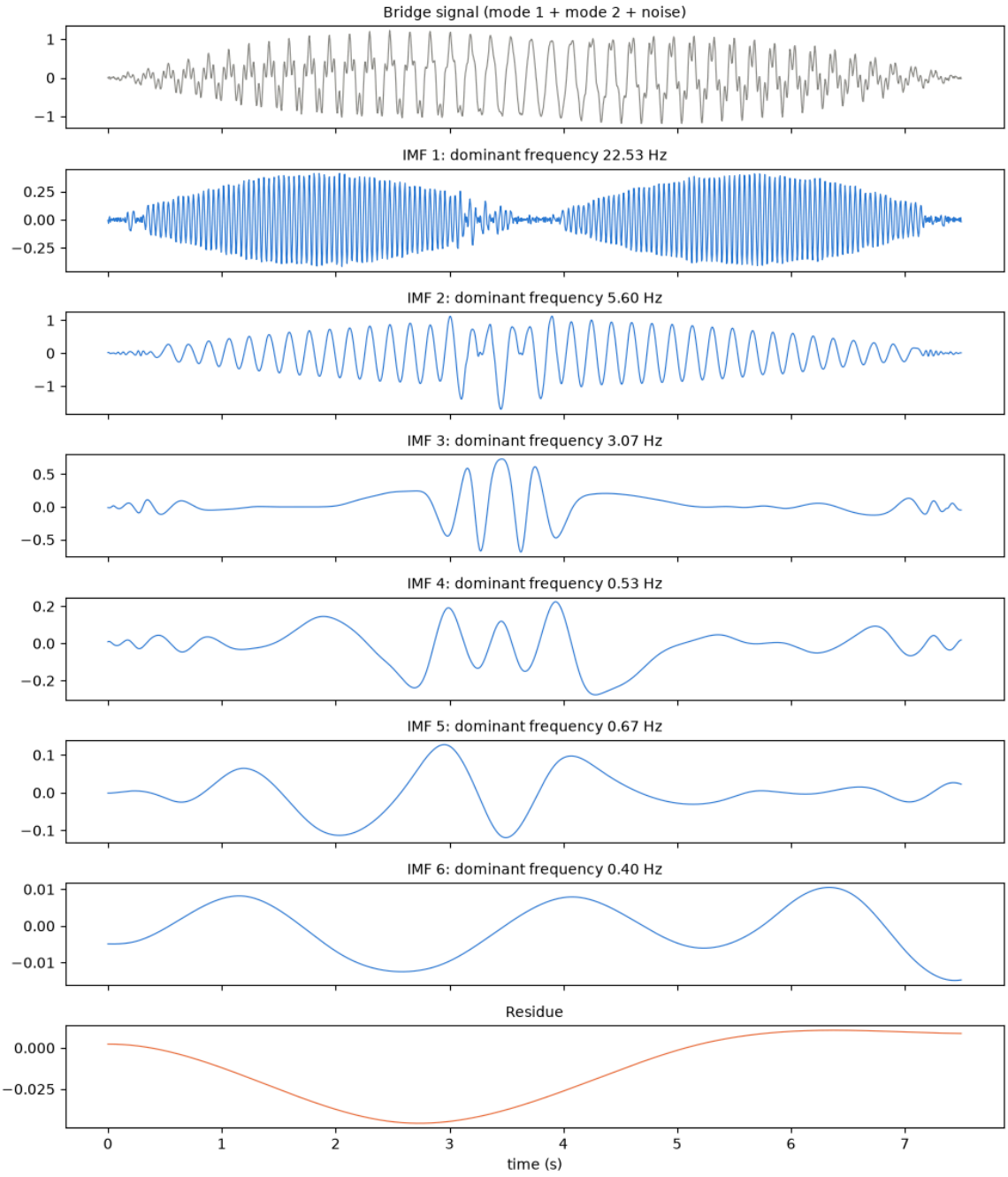
Hay un máximo y un mínimo por cada ciclo de la oscilación rápida. La oscilación lenta no crea picos propios: solo sube y baja la altura a la que caen los picos de la rápida.

En Python se puede usas la librería `PyEMD` para aplicar el método y descomponer una señal en sus IMFs.

Limitantes:

- Mezcla de modos: cuando dos oscilaciones vibran a frecuencias cercanas, el EMD puede mezclar ambas en una sola IMF.
- Modos repartidos en diferentes IMFs: cuando una oscilacion es intermietente, puede provocar que 1 modo de vibracion se reparta en dos IMFs.
- Malekjafarian et. al (2017) demostraron que si la excitación hace entrar en resonancia el modo que se busca, ese modo domina la señal y la EMD lo separa fácilmente en una IMF.

Example:



2. PSD from the autocorrelation and the Fourier transform

- Basado en el teorema de Wiener-Khinchin (Wiener, 1930; Khinchin, 1934): la PSD es la transformada de Fourier de la autocorrelación.
- Visto en CIV E 789: autocorrelación en la sesión 5 (Lecture 3, diapositivas 20 a 23) y la PSD como transformada de la autocorrelación en la sesión 7 (Lecture 4, diapositiva 22).
- La autocorrelación mide cuánto se parece la señal a sí misma desplazada un tiempo τ. Al pasarla al dominio de la frecuencia con la transformada de Fourier, se obtiene la PSD.

$$S_{xx}(f) = \int_{-\infty}^{\infty} R_{xx}(\tau)\, e^{-j2\pi f\tau}\, d\tau$$

- $S_{xx}(f)$: PSD de la señal x en la frecuencia f.
- $R_{xx}(\tau)$: autocorrelación de x para un desfase τ.
- $e^{-j2\pi f\tau}$: la transformada de Fourier, que compara la autocorrelación con una oscilación de frecuencia f.

Process:

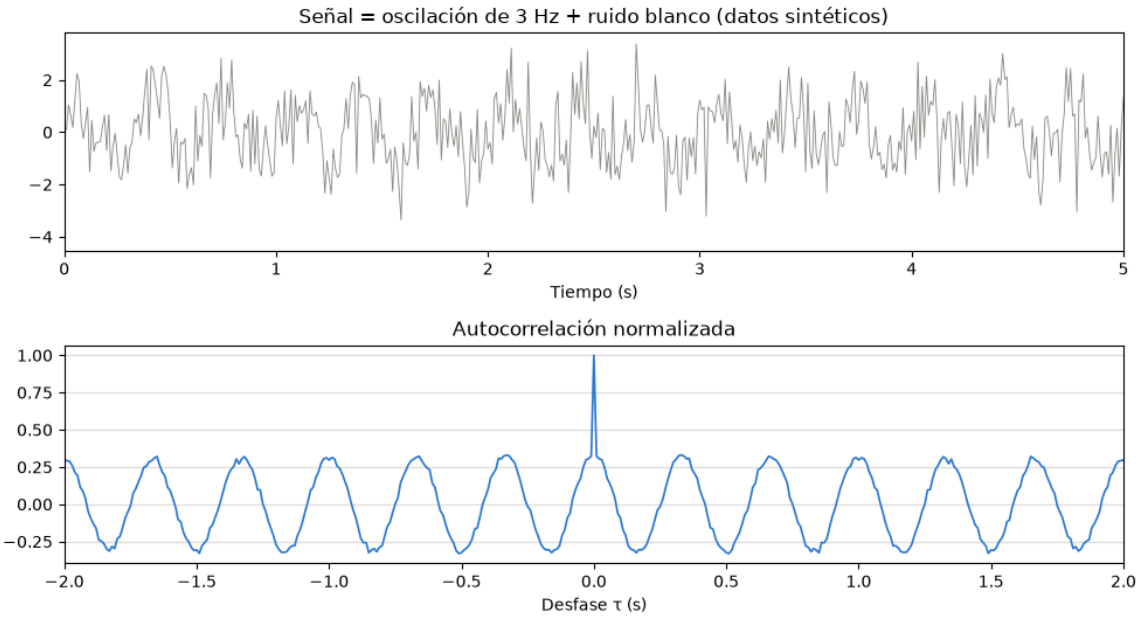
1. Remove the mean of the signal.
2. Compute the autocorrelation for all time lags.
3. Compute the Fourier transform of the autocorrelation.

```
fs = 100.0          # sampling frequency (Hz)
duration = 20.0     # signal length (s)
f0 = 3.0            # tone frequency (Hz)

rng = np.random.default_rng(0)
t = np.arange(0, duration, 1 / fs)
x = np.sin(2 * np.pi * f0 * t) + 1.0 * rng.standard_normal(t.size)
x = x - x.mean()

# Biased autocorrelation (divided by N), normalized so that R(0) = 1
r = np.correlate(x, x, mode="full") / x.size
lags = np.arange(-x.size + 1, x.size) / fs
r = r / r[x.size - 1]

fig, (ax_sig, ax_r) = plt.subplots(2, 1, figsize=(10, 5.5))
ax_sig.plot(t, x, color="#8a8984", lw=0.6)
ax_sig.set_title("Señal = oscilación de 3 Hz + ruido blanco (datos sintéticos)")
ax_sig.set_xlabel("Tiempo (s)")
ax_sig.set_xlim(0, 5)
ax_r.plot(lags, r, color="#2a78d6", lw=1.2)
ax_r.set_title("Autocorrelación normalizada")
ax_r.set_xlabel("Desfase τ (s)")
ax_r.set_xlim(-2, 2)
ax_r.grid(axis="y", color="#e1e0d9")
plt.tight_layout()
plt.show()
```



En la señal casi no se ve la oscilación, pero en la autocorrelación sí: el ruido blanco solo aporta un pico en $\tau = 0$, mientras que la oscilación de 3 Hz deja un coseno que se repite cada 0.33 s. Su transformada de Fourier da un pico en 3 Hz.

En Python, la autocorrelación se calcula con `np.correlate(x, x, mode="full")` o `scipy.signal.correlate`, y la transformada con `np.fft.fft`.

Usos principales:

- Obtener la PSD y, con ella, las frecuencias naturales (los picos).
- Con la correlación cruzada entre sensores se obtienen las PSD cruzadas que usa la FDD.
- Es la base de métodos de identificación con vibración ambiental que trabajan con correlaciones, como NExT.

Limitantes:

- El resultado es exactamente el periodograma ($|FFT|^2$ de toda la señal), así que fluctúa igual.
- Los desfases grandes se calculan con pocas muestras y son poco confiables.

Example:

```
# Colors (same roles as in metodos.ipynb)
COLOR_SIGNAL = "#8a8984"   # raw / measured signal (neutral gray)
COLOR_1 = "#2a78d6"        # main series (blue)
COLOR_2 = "#eb6834"        # second series (orange)

# Shared example: vehicle crossing a simply supported bridge
L_SPAN = 15.0          # span (m)
V_SPEED = 2.0          # vehicle speed (m/s)
F1, F2 = 5.65, 22.62   # bridge natural frequencies (Hz)
FS = 200.0             # sampling frequency (Hz)

t = np.arange(0, L_SPAN / V_SPEED, 1 / FS)
```

```
position = V_SPEED * t                          # vehicle position x = v t (m)
phi_1 = np.sin(np.pi * position / L_SPAN)       # mode 1 shape
phi_2 = np.sin(2 * np.pi * position / L_SPAN)   # mode 2 shape

rng = np.random.default_rng(42)   # fixed seed for the noise

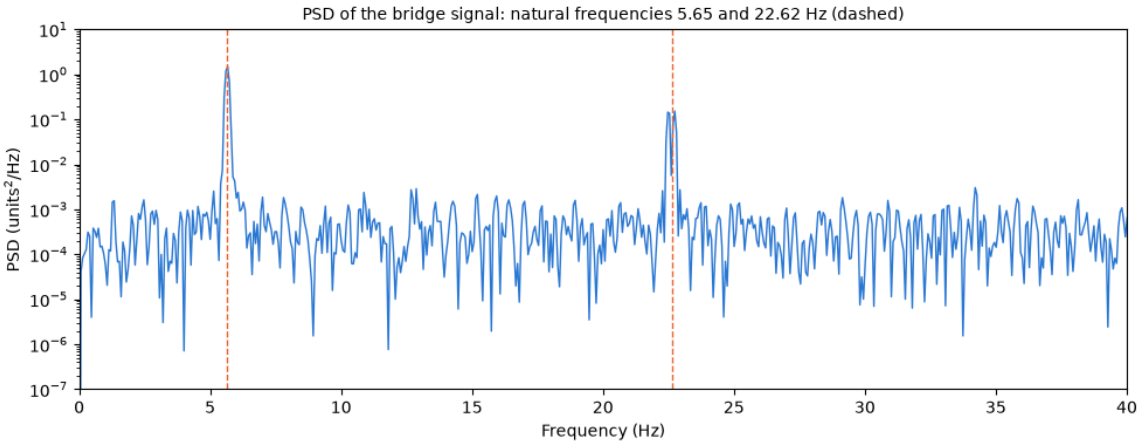
# Bridge signal: mode 1 + mode 2 + noise
x_bridge = (phi_1 * np.cos(2 * np.pi * F1 * t)
            + 0.4 * phi_2 * np.cos(2 * np.pi * F2 * t)
            + 0.2 * rng.standard_normal(t.size))
x_bridge = x_bridge - x_bridge.mean()
N = x_bridge.size


def psd_from_autocorr(r_full, fs, n_fft):
    # One-sided PSD = Fourier transform of the autocorrelation (lags -(N-1)...(N-1))
    r_circ = np.fft.ifftshift(r_full)               # put lag 0 first
    s = np.real(np.fft.fft(r_circ, n_fft)) / fs     # two-sided PSD (units^2/Hz)
    f = np.fft.rfftfreq(n_fft, 1 / fs)
    s_one = s[:f.size].copy()
    last = -1 if n_fft % 2 == 0 else None           # Nyquist bin exists only for even
n_fft
    s_one[1:last] *= 2                              # one-sided: double all bins except
0 Hz and Nyquist
    return f, s_one


r_full = np.correlate(x_bridge, x_bridge, mode="full") / N   # biased autocorrelation
n_fft = 2 * N - 1

# PSD = Fourier transform of the autocorrelation
f_ac, s_ac = psd_from_autocorr(r_full, FS, n_fft)

fig, ax = plt.subplots(figsize=(10, 4))
ax.semilogy(f_ac, s_ac, color=COLOR_1, lw=1)
for f_n in (F1, F2):
    ax.axvline(f_n, color=COLOR_2, ls="--", lw=1)
ax.set_title("PSD of the bridge signal: natural frequencies 5.65 and 22.62 Hz
(dashed)", fontsize=10)
ax.set_xlabel("Frequency (Hz)")
ax.set_ylabel("PSD (units$^2$/Hz)")
ax.set_xlim(0, 40)
ax.set_ylim(1e-7, 10)
fig.tight_layout()
plt.show()
```



Los picos de la PSD aparecen en las frecuencias naturales del puente (5.65 y 22.62 Hz). El piso entre los picos corresponde al ruido.

3. Power Spectral Density (PSD) — Welch's method

- Proposed by Welch (1967) in "The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging Over Short, Modified Periodograms."
- Estima cómo se reparte la energía de la señal entre las frecuencias. En vez de una sola FFT de toda la señal, promedia las FFT de varios segmentos para obtener un espectro más estable.

Process:

1. Divide the signal into segments with 50 % overlap.
2. Multiply each segment by a Hann window.
3. Compute the FFT of each segment and its power $|X|^2$.
4. Average the powers of all segments.

```
fs = 100.0              # sampling frequency (Hz)
duration = 20.0         # signal length (s)
n_seg = 512             # samples per segment
step = n_seg // 2       # 50 % overlap

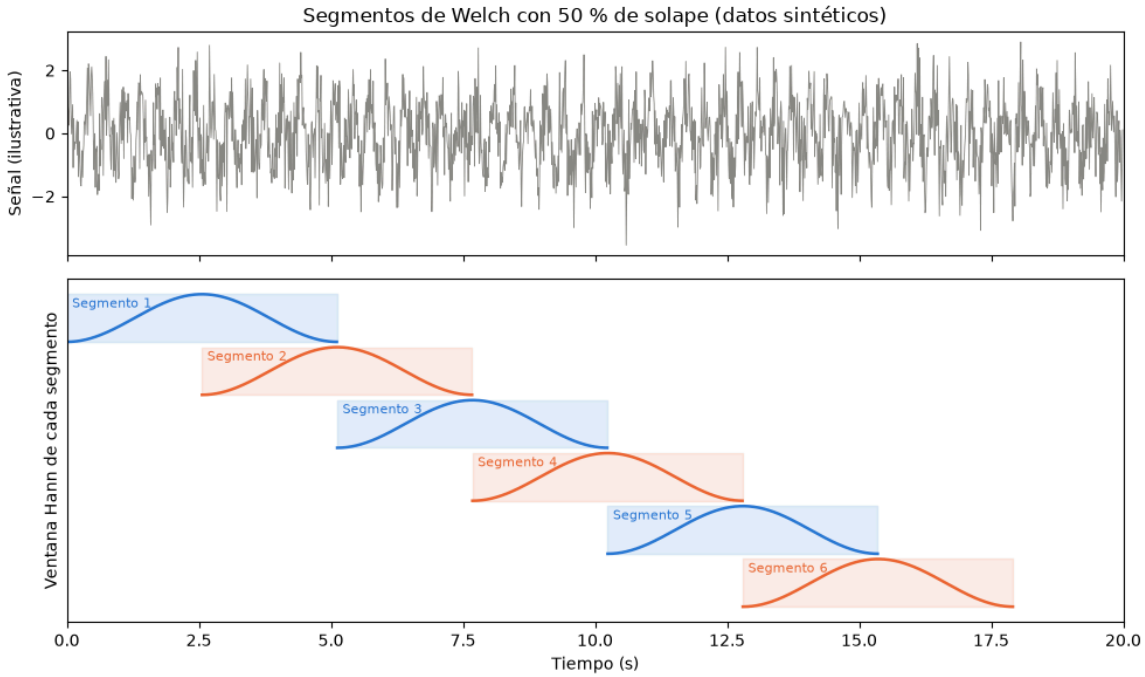
rng = np.random.default_rng(0)
t = np.arange(0, duration, 1 / fs)
x = np.sin(2 * np.pi * 3.0 * t) + 0.8 * rng.standard_normal(t.size)

starts = np.arange(0, t.size - n_seg + 1, step)
hann = get_window("hann", n_seg)

fig, (ax_sig, ax_win) = plt.subplots(2, 1, figsize=(10, 6), sharex=True,
                                     gridspec_kw={"height_ratios": [1.3, 2]})
ax_sig.plot(t, x, color="#8a8984", lw=0.6)
ax_sig.set_title("Segmentos de Welch con 50 % de solape (datos sintéticos)")
ax_sig.set_ylabel("Señal (ilustrativa)")

# One row per segment: shaded span = samples used, curve = Hann window applied to them
for i, s in enumerate(starts):
    row = starts.size - 1 - i
```

```
    color = ["#2a78d6", "#eb6834"][i % 2]
    seg_t = t[s:s + n_seg]
    ax_win.axhspan(row, row + 0.9, xmin=seg_t[0] / duration, xmax=seg_t[-1] /
duration,
                   color=color, alpha=0.12)
    ax_win.plot(seg_t, row + 0.9 * hann, color=color, lw=1.8)
    ax_win.text(seg_t[0] + 0.1, row + 0.65, f"Segmento {i + 1}", fontsize=8,
color=color)
ax_win.set_yticks([])
ax_win.set_ylabel("Ventana Hann de cada segmento")
ax_win.set_xlabel("Tiempo (s)")
ax_win.set_xlim(0, duration)
plt.tight_layout()
plt.show()
```



Cada fila es un segmento: la franja muestra las muestras que usa y la curva, la ventana Hann que se les aplica. Cada segmento empieza en la mitad del anterior (50 % de solape), así que cada instante entra en dos segmentos, y la ventana hace que los bordes de cada segmento pesen poco.

En Python se puede usar `scipy.signal.welch` para calcular la PSD.

Usos principales:

- Identificar las frecuencias naturales de la estructura (los picos), sobre todo con vibración ambiental.
- Estimar el amortiguamiento a partir del ancho de los picos (half-power bandwidth).
- Calcular PSD cruzadas entre sensores para obtener las formas modales (FDD), la FRF y la coherencia.
- Monitorear cambios en las frecuencias o en la energía de cada modo como indicador de daño.

Limitantes:

- Compromiso entre resolución y estabilidad: segmentos más cortos dan más promedios y un espectro más suave, pero los picos se ensanchan ($\Delta f = f_s / N_{seg}$).
- Supone una señal estacionaria: si el contenido cambia en el tiempo, pierde cuándo ocurrió (para eso están el STFT y las wavelets).
- Promediar reduce la fluctuación de la estimación, no el ruido de la señal.

Welch vs. FFT de toda la señal:

	FFT de toda la señal	PSD con Welch
Resolución en frecuencia	Mejor (separa picos cercanos)	Menor ($\Delta f = f_s / N_{seg}$)
Estabilidad del espectro	Fluctúa mucho y no mejora con registros más largos	Más estable al promediar segmentos
Picos falsos por ruido	Frecuentes	Menos, los picos reales se distinguen mejor
Depende de la duración del registro	Sí (la magnitud cambia con el número de muestras)	No (se normaliza por hertz)
Coherencia y PSD cruzadas	Sin sentido (con un solo bloque la coherencia da 1)	Confiables

Example:

```
# Colors (same roles as in metodos.ipynb)
COLOR_SIGNAL = "#8a8984"   # raw / measured signal (neutral gray)
COLOR_1 = "#2a78d6"        # main series (blue)
COLOR_2 = "#eb6834"        # second series (orange)

# Shared example: vehicle crossing a simply supported bridge
L_SPAN = 15.0          # span (m)
V_SPEED = 2.0          # vehicle speed (m/s)
F1, F2 = 5.65, 22.62   # bridge natural frequencies (Hz)
FS = 200.0             # sampling frequency (Hz)

t = np.arange(0, L_SPAN / V_SPEED, 1 / FS)
position = V_SPEED * t                          # vehicle position x = v t (m)
phi_1 = np.sin(np.pi * position / L_SPAN)       # mode 1 shape
phi_2 = np.sin(2 * np.pi * position / L_SPAN)   # mode 2 shape

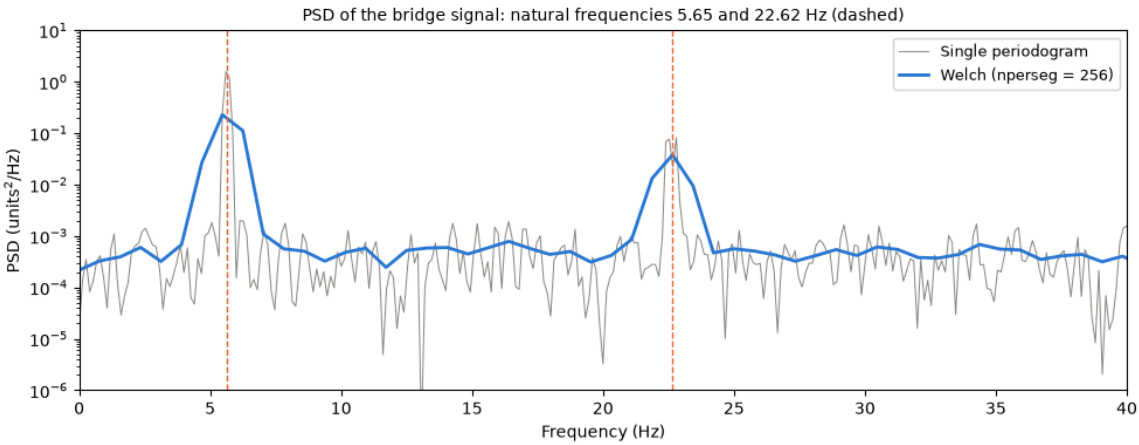
rng = np.random.default_rng(42)   # fixed seed for the noise

# Bridge signal: mode 1 + mode 2 + noise
x_bridge = (phi_1 * np.cos(2 * np.pi * F1 * t)
            + 0.4 * phi_2 * np.cos(2 * np.pi * F2 * t)
            + 0.2 * rng.standard_normal(t.size))

f_per, p_per = periodogram(x_bridge, FS, window="hann")        # single FFT of the
whole record
f_w, p_w = welch(x_bridge, FS, window="hann", nperseg=256)     # Welch average

fig, ax = plt.subplots(figsize=(10, 4))
```

```
ax.semilogy(f_per, p_per, color=COLOR_SIGNAL, lw=0.7, label="Single periodogram")
ax.semilogy(f_w, p_w, color=COLOR_1, lw=2, label="Welch (nperseg = 256)")
for f_n in (F1, F2):
    ax.axvline(f_n, color=COLOR_2, ls="--", lw=1)
ax.set_title("PSD of the bridge signal: natural frequencies 5.65 and 22.62 Hz
(dashed)", fontsize=10)
ax.set_xlabel("Frequency (Hz)")
ax.set_ylabel("PSD (units$^2$/Hz)")
ax.set_xlim(0, 40)
ax.set_ylim(1e-6, 10)
ax.legend(loc="upper right", fontsize=9)
fig.tight_layout()
plt.show()
```



El periodograma de toda la señal tiene picos más angostos, pero su piso de ruido fluctúa mucho. Welch da un espectro más estable, a cambio de picos más anchos.